

Adopting Agile Methods for Follow-the-Sun Software Development

Josiane Kroll

Computer Science School, PUCRS, Brazil
josiane.kroll@acad.pucrs.br

Jorge Luis Nicolas Audy

Computer Science School, PUCRS, Brazil
audy@pucrs.br

ABSTRACT

Follow-the-sun (FTS) is a special case of Global Software Development (GSD) in which development is distributed around twenty-four hours of work per day in order to reduce the overall project completion time. However, FTS is barely used by software development companies as they find it difficult to put FTS into practice. Recent studies have suggested agile methods as a promising approach to implement FTS. Thus, in this study we examine how agile methods can collaborate with FTS. We present an experience report describing the challenges and lessons learned by applying the Scrum method to a software development project. Moreover, we discuss how agile methods can be used effectively for FTS software development aiming to target the project managers who want to better reap the benefits of FTS.

Keywords

Global software development, follow-the-sun, agile methods, software engineering.

INTRODUÇÃO

O desenvolvimento global de software (*Global Software Development*¹ – GSD) permite que novas abordagens sejam introduzidas para o desenvolvimento de projetos de software na qual recursos, investimentos, usuários e equipes de desenvolvimento podem estar distribuídos em diferentes locais e fusos horários (Sato, Huzita e Leal, 2011). Além disso, o GSD também permite que empresas criem subsidiárias em outros países e configurem a distribuição de processos de software nas vinte e quatro horas de um dia de trabalho (Casey, Deshpande e Richardson, 2008). Essa estratégia usada para o desenvolvimento de software vinte e quatro horas é chamada de *follow-the-sun* (FTS).

O desenvolvimento de software FTS ainda está nos estados iniciais de pesquisa e são encontrados muitos desafios relacionados à comunicação, coordenação e cultura para a sua prática (Hess e Audy, 2012). Segundo Carmel, Espinosa e Dubinsky (2010) há poucos casos de sucesso documentados com a aplicação de FTS na indústria de software e faltam estudos sobre as suas principais características.

Os métodos ágeis são discutidos na literatura como uma forma promissora para a aplicação de FTS (Gupta et al., 2012; Hess e Audy, 2012; Carmel, Espinosa e Dubinsky, 2010; Yap, 2005). Além disso, práticas ágeis estão cada vez mais sendo aplicadas para equipes de desenvolvimento colocalizados (Gupta et al., 2012). Nesse sentido, este estudo objetiva examinar como os métodos ágeis podem colaborar para o desenvolvimento de software FTS.

Para alcançar esse objetivo é investigada a base teórica de FTS, desafios enfrentados e soluções propostas para minimizá-los. Em seguida, é apresentada a experiência obtida com a aplicação do método Scrum para o desenvolvimento de um projeto de software no modo FTS.

Este artigo está estruturado da seguinte forma: na seção Contextualização são apresentados os principais temas envolvidos neste estudo, desenvolvimento de software *follow-the-sun* e métodos ágeis. Na seção Histórico da Aplicação dos Métodos Ágeis para FTS, são apresentados os principais estudos que abordam o uso dos métodos ágeis para FTS e os resultados obtidos. Na seção Estudo de Caso é reportada a experiência obtida com a aplicação do método Scrum para o desenvolvimento de um projeto de software no modo FTS. Na seção Discussão é apresentada uma análise crítica dos resultados obtidos. Por fim, a Conclusão mostra as principais contribuições desse estudo e os trabalhos futuros.

¹ O termo será mantido em inglês, visto que a tradução em alguns casos pode comprometer o uso do termo.

CONTEXTUALIZAÇÃO

Desenvolvimento de Software *Follow-the-sun*

Follow-the-sun (FTS) é uma estratégia de desenvolvimento de software aplicada para o contexto de projetos de GSD (Carmel, Espinosa e Dubinsky, 2010). Ela se caracteriza pelo desenvolvimento de software vinte e quatro horas contínuas com equipes geograficamente distribuídas (Visser e Solingen, 2009).

Em FTS, os membros de uma equipe são distribuídos de maneira que quando um local de produção encerra o seu dia de trabalho, membros da equipe localizados em um diferente local de produção e fuso horário assumem as tarefas iniciando a sua jornada de trabalho (Treinen e Miller-Frost, 2006). O desenvolvimento FTS considera as diferenças de tempo e a comunicação contínua para definir turnos e horários de trabalho, com reuniões regulares e relatórios de progresso, a fim de identificar e aplicar estratégias adequadas do início ao final do projeto (Carmel e Espinosa, 2011).

A produção diária gerada por uma equipe FTS é enviada para o próximo local de produção, o qual está em um diferente fuso horário para dar continuidade ao trabalho (Visser e Solingen, 2009). A continuidade do trabalho envolve ciclos de troca de tarefas entre as equipes que são separados por *handoffs* diários (Solingen e Valkema, 2010). O *handoff* é um termo utilizado na literatura para designar o processo de transição de tarefas entre equipes de unidade de trabalho para o próximo local de produção (Carmel, Dubinsky e Espinosa, 2009).

O principal objetivo da estratégia FTS é reduzir o ciclo de desenvolvimento do projeto ou *time-to-market*. Segundo Carmel, Dubinsky e Espinosa (2009), a estratégia FTS não oferece outras vantagens além da redução do tempo de desenvolvimento do projeto. Dessa forma, a estratégia FTS se diferencia do GSD tradicional quando quatro critérios são satisfeitos:

1. *O principal objetivo é aumentar a velocidade do desenvolvimento reduzindo o tempo de duração do projeto.* Esse critério distingue o FTS das outras práticas e configurações do GSD tradicional. O FTS não oferece outras vantagens sobre outras configurações além da velocidade.
2. *Os locais de produção são distantes e em diferentes fusos horários.* Esse critério diferencia o FTS do GSD tradicional pela aceleração das táticas de produção.
3. *Em qualquer ponto no tempo apenas um local de produção possui o produto final (inacabado).* Esse critério diferencia o FTS a partir das configurações convencionais globais em que vários locais de produção podem possuir diferentes partes do produto.
4. *Handoffs são realizados diariamente no final de cada turno de trabalho.* Este critério diferencia o FTS das configurações convencionais globais que minimiza as dependências de *handoffs* entre os locais de produção.

Em FTS, são identificados centros que podem especializar-se em fases bem definidas do ciclo de desenvolvimento de software. Há uma tendência do FTS concentrar-se nas fases de implementação permitindo que artefatos possam ser transferidos entre locais de desenvolvimento. No entanto, devido à diversidade entre estes locais, podem existir muitas dificuldades relacionadas à comunicação, coordenação e cultura para a aplicação do FTS (Cameron, 2004).

Na literatura, muitas vezes FTS também é referenciado por meio de conceitos similares tais como, *24-hour development model*, *24-Hour Knowledge Factory Paradigm* (24HrKF) e *round-the-clock*. Embora esses termos sejam usados como sinônimos na literatura, eles possuem diferentes definições.

O conceito de FTS é focado na velocidade e na redução da duração do projeto, enquanto o conceito *round-the-clock* e outros, abordam o desenvolvimento de software 24 horas, mas executando operações em todos os turnos de trabalho. Todos esses conceitos usam as diferenças de fusos horários para projetar turnos de trabalho, mas para diferentes propósitos e com diferentes tipos de tarefas (Carmel e Espinosa, 2011).

Métodos Ágeis

Os métodos ágeis, também conhecidos como métodos adaptativos, tem o propósito de superar as limitações de metodologias de desenvolvimento de software que seguem um planejamento rígido considerando as mudanças dos requisitos do sistema. Os métodos ágeis estão focados no estabelecimento e colaboração entre clientes e desenvolvedores e na entrega do software dentro do prazo e sem restrições de orçamento (Smite, Moe e Agerfalk, 2010).

As práticas de desenvolvimento de software ágil enfatizam a necessidade de comunicação entre equipes de desenvolvimento e entre equipes e clientes do projeto. Essas práticas procuram resolver muitas questões da engenharia do software, tais como, custos mais elevados que os previstos e requisitos não atendidos (Niinimäki, 2011).

No desenvolvimento de software ágil as equipes são motivadas por benefícios como o aumento da produtividade, inovação e satisfação profissional. Já para a empresa a adoção do desenvolvimento de software ágil é motivada pelo aumento da complexidade do desenvolvimento de software (Smite, Moe e Agerfalk, 2010).

Os métodos ágeis tem foco em pequenos intervalos de tempo para atualização durante o desenvolvimento de software. Trata-se de altos níveis de colaboração com menos tempo para a documentação (Smite, Moe e Agerfalk, 2010). Os métodos ágeis baseiam-se na engenharia robusta da própria organização das equipes, as quais são adaptáveis às mudanças (Sutherland et al., 2007).

Na literatura os métodos ágeis mais discutidos para o desenvolvimento de projetos de software distribuídos são o Scrum e o *Extreme Programming* (XP) (Deny et al., 2008). Segundo Paasivaara, Durasiewicz e Lassenius (2008), métodos ágeis tais como Scrum e XP, são os mais recomendados para projetos distribuídos, porque podem ser customizados com sucesso.

O método Scrum é definido como um *framework* de desenvolvimento de software ágil (Bannerman, Hossain e Jeffery, 2012). Ele permite aumentar a velocidade do desenvolvimento, alinhar os objetivos individuais e da organização, criar uma cultura impulsionada pela criação do desempenho, conseguir uma comunicação estável e consistente de desempenho em todos os níveis e promover o desenvolvimento individual (Sutherland et al., 2007).

Já o XP é um método que está focado na comunicação eficaz entre a equipe. Seu uso é recomendado para equipes pequenas que necessitam desenvolver software de forma rápida e em um ambiente de rápida evolução de requisitos (Yap, 2005). As práticas do método XP não incluem a elaboração de requisitos extensos ou a documentação do projeto antes do início do desenvolvimento. No entanto, o XP proporciona uma comunicação contínua entre as partes interessadas (Kircher et al., 2011).

Na Tabela 1 é apresentado um quadro comparativo baseado nas principais características dos métodos Scrum e XP.

Características	Scrum	XP
Visa oferecer a comunicação efetiva entre os membros da equipe		x
Baseado nos princípios de simplicidade e <i>feedback</i>		x
Projetado para o uso de pequenas equipes		x
Objetiva acelerar o desenvolvimento de software	x	x
É recomendado para ambientes de rápida mudança de requisitos		x
Pré-versão do produto pode ser usada pelos clientes	x	
Baseado no <i>feedback</i> dos clientes		x
Não inclui a preparação extensiva de requisitos ou <i>design</i> de documentos	x	
Não gera documentação	x	x
Reuniões diárias	x	x
Pequenas interações	x	
Uso iterativo de versões	x	
Prioriza tarefas e o desenvolvimento dirigido por testes (ou TDD)		x
Prioriza itens (arquivo <i>backlog</i>)	x	
Trabalho baseado em iterações (2- 3 semanas)	x	
Trabalho baseado em iterações (1- 2 semanas)		x
Suscetível a mudanças		x
Framework incremental	x	x
Possui técnica de gerenciamento do projeto	x	

Tabela 1. Quadro Comparativo entre os Métodos Scrum e XP

Pode-se observar na Tabela 1 que as características semelhantes entre os métodos Scrum e XP são comuns à maioria dos métodos para desenvolvimento de software ágil. Métodos ágeis estão focados no contínuo levantamento de requisitos, frequente comunicação, contínua integração, *feedback* do cliente para especialistas e pouca documentação (Jalali e Wohlin, 2010).

Já as características que diferenciam um método do outro estão direcionados aos objetivos específicos para quais estes métodos foram concebidos. O XP visa principalmente oferecer uma efetiva comunicação entre os membros da equipe (Layman et al., 2006). Por outro lado o Scrum visa o rápido desenvolvimento e entrega do produto (Paasivaara, Durasiewicz e Lassenius, 2008).

Histórico da Aplicação dos Métodos ágeis para FTS

Estudos sobre o desenvolvimento de software FTS são desenvolvidos desde 1996 (Carmel, 1999). Entretanto, a dificuldade em encontrar empresas adotando plenamente FTS como estratégia de desenvolvimento de software é discutida como a principal barreira para a evolução de FTS na indústria de software. Dessa forma, a aplicação dos métodos ágeis para FTS ainda tem sido pouco explorada. A seguir são apresentados os principais estudos que discutem a aplicação dos métodos ágeis para FTS e os resultados obtidos.

- *Kroll et al. (2012)*

Este estudo investigou os benefícios das abordagens adaptativas e prescritivas para FTS. Resultados obtidos neste estudo indicam que o uso das abordagens adaptativas pode aumentar a velocidade, mas não melhora a precisão e a qualidade do trabalho realizado pelas equipes. Além disso, baseado nos resultados encontrados, os autores sugerem que as abordagens adaptativas são mais apropriadas do que as abordagens prescritivas para o contexto de FTS, colaborando com o estudo realizado por Carmel, Espinosa e Dubinsky (2010).

- *Carmel, Espinosa e Dubinsky (2009)*

Neste estudo foi desenvolvido um quasi-experimento designado a testar FTS mensurando a velocidade de trabalho de equipes distribuídas de acordo com uma abordagem de desenvolvimento ágil. Resultados obtidos mostraram que as equipes que usaram a estratégia FTS terminaram suas tarefas mais rápido do que equipes colocalizadas, devido ao uso das práticas ágeis. O tempo ganho foi de 10%, mas os autores deste estudo acreditam que o ganho poderia ser ainda maior.

- *Deny et al. (2008)*

No estudo realizado por Deny et al. (2008) foi explorada a utilização de práticas ágeis para ambientes de desenvolvimento 24 horas, com o propósito de permitir que *handoffs* ocorressem de maneira eficaz. O desenvolvimento deste estudo é motivado pelo interesse no desenvolvimento de novos processos de software ágeis e distribuídos, que estejam especialmente voltados para o uso em um ambiente de desenvolvimento de software 24 horas.

- *Yap (2005)*

Este estudo relata a experiência da aplicação do XP distribuído para FTS. A autora descreve os desafios que são enfrentados nesse ambiente, lições aprendidas e como resolver questões como a integração contínua global, diferenças culturais e prioridades conflitantes entre os locais de desenvolvimento. Os resultados obtidos nesse estudo mostram que o XP é indicado para FTS.

ESTUDO DE CASO

Nesta seção são apresentados detalhes do desenvolvimento de um estudo em uma grande empresa de desenvolvimento de software global. A empresa em questão fornece serviços de consultoria de negócios, tecnologia, serviços de engenharia e *outsourcing* para clientes distribuídos em mais de 32 países.

O desenvolvimento do estudo de caso foi realizado entre os meses de maio e agosto de 2012 e constou do desenvolvimento de um projeto de software interno da empresa no modo FTS. Este estudo focou na fase de desenvolvimento de um projeto de software. Para o desenvolvimento do projeto foi adotado o método Scrum. A escolha do método Scrum foi realizada pela empresa, a qual possuía interesse em explorar esse método.

A equipe alocada para o projeto foi composta de seis desenvolvedores e um gerente de projeto. O desenvolvimento do projeto foi estimado para aproximadamente um mês de duração.

Configuração da equipe

A equipe alocada para o desenvolvimento do projeto de software foi distribuída em três diferentes locais de produção e fusos horários (Figura 1). Dois desenvolvedores no México, dois desenvolvedores na Austrália e outros dois desenvolvedores e o gerente do projeto na Índia.

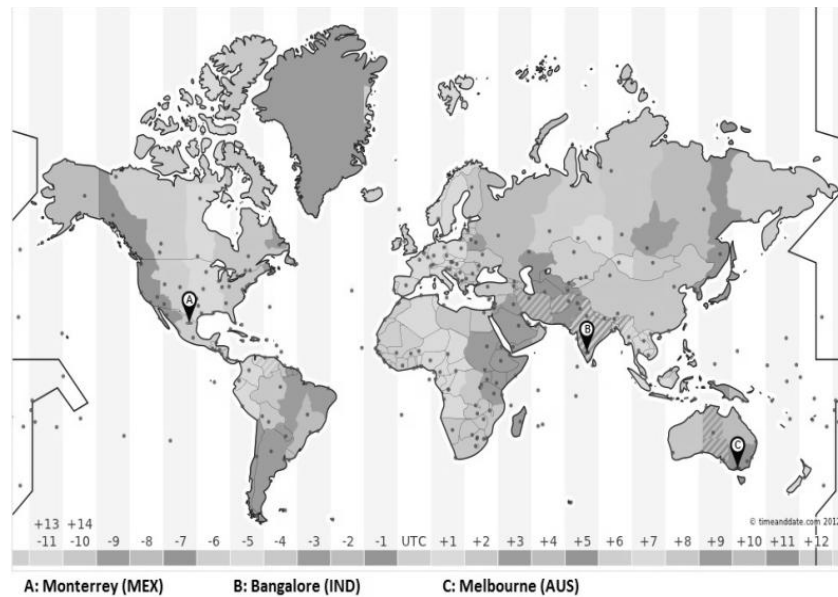


Figura 1. Distribuição da Equipe FTS em Diferentes Locais e Fusos Horários

No México, os dois desenvolvedores eram novas contratações da empresa e estavam em período de treinamento. Os dois desenvolvedores da Índia tinham dois anos e um ano de experiência, respectivamente. O gerente do projeto, embora tendo experiência de trabalho de aproximadamente 10 anos, ainda não possuía experiência como gerente de projeto. Os desenvolvedores da Austrália eram os mais experientes, entre 8 e 14 anos de experiência.

A equipe alocada para o projeto não possuía experiência em FTS e somente os desenvolvedores da Austrália tinham experiência em métodos ágeis. Para minimizar o problema de falta de experiência, sessões de treinamento foram realizadas para esclarecer questões sobre o desenvolvimento de software FTS, método *Scrum* e a abordagem adotada para o desenvolvimento das atividades do projeto. Além disso, um *Scrum master* foi alocado para o projeto para assegurar que as práticas *Scrum* seriam seguidas.

Desenvolvimento do projeto

O desenvolvimento do projeto foi dividido em dois *sprints*. Cada *sprint* com duas semanas de duração.

Para realizar as sessões de *handoff* optou-se pela realização de chamadas telefônicas no final e no início de cada dia de trabalho. Cada ligação deveria durar no máximo 30 minutos.

Para que as tarefas fossem transferidas de um local de produção para o outro, foi criado um *template* usando o software Microsoft Excel. Esse *template* foi disponibilizado no portal do projeto. O *template* foi criado baseado no estudo realizado por Hess e Audy (2012). O portal do projeto também serviu de repositório de código e de outros documentos do projeto.

A alocação das tarefas seguiu o conceito CPro. O CPro é um processo de software ágil que considera a distribuição de tarefas para CPs (*Composite Persona*). As tarefas são alocadas para CPs. Cada CP é formado por um desenvolvedor de cada localização, onde a alocação ocorre verticalmente (Deny et al., 2008) (Figura 2).

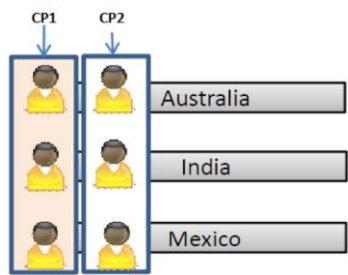


Figura 2. Alocação de Tarefas baseado em CPs

Desafios enfrentados

Muitos desafios foram enfrentados principalmente no início do projeto. O principal desafio enfrentado foi o planejamento do projeto. A equipe alocada para o projeto não tinha prévia experiência com o desenvolvimento de projetos FTS. Também tinha pouca ou nenhuma experiência com o método Scrum. Dessa forma, o planejamento do projeto tentou seguir padrões e práticas já adotados pela empresa. Entretanto, cabe ressaltar que os padrões e práticas de software adotados pela empresa não atendiam todas as características do desenvolvimento FTS. Dessa forma, a equipe usou a experiência adquirida com projetos anteriores para planejar o projeto.

As diferenças de fusos horários foi outro desafio enfrentado tanto na fase de planejamento como na fase de execução do projeto. Os membros da equipe estavam distribuídos em três fusos horários diferentes e foi necessário ajustar as horas de trabalho da equipe para conseguir um bom *overlap*. Segundo Deshpande e Richardson (2009) para que FTS seja implementado é necessário pelo menos uma hora de *overlap* entre cada local de produção.

O gerenciamento de fusos horários possibilitou que sessões de *handoff* fossem realizadas entre os locais de produção. Entretanto, durante as sessões de *handoff* muitos problemas de coordenação foram encontrados. As primeiras sessões de *handoff* ultrapassaram os 30 minutos planejados e membros da equipe não se sentiam confortáveis para fazer perguntas e solicitavam emails complementares com o que já havia sido discutido.

As diferenças culturais e de comunicação também causaram desafios para o desenvolvimento do projeto. Os desafios culturais estavam associados à diversidade sociocultural, determinada principalmente por aspectos sociais, étnicos e religiosos da equipe alocada para o projeto. A comunicação entre os membros da equipe era difícil de ser realizada devido aos diferentes sotaques e a linguagem técnica, a qual não era comum a todos os membros da equipe.

No projeto FTS outro desafio enfrentado foi a alocação de tarefas. As tarefas foram primeiramente disponibilizadas para a equipe no arquivo *sprint backlog*, mas os membros da equipe se sentiam confusos para categorizar as tarefas apropriadamente. Muitas tarefas não foram realizadas ou então iniciadas e não completadas pela equipe.

Lições aprendidas

O desenvolvimento do estudo de caso contribuiu para identificar algumas lições aprendidas, as quais são apresentadas a seguir:

- *Sprint backlog*

Um email diário alocando tarefas individualmente para cada dia de trabalho contribui para definir prioridades de tarefas e reduzir problemas enfrentados pela equipe para categorizar uma tarefa dentro de um arquivo *sprint backlog*.

- Elegir um responsável pela tarefa

Algumas tarefas durante o desenvolvimento do projeto foram atribuídas para um desenvolvedor responsável. Observou-se que a eleição de um responsável pela tarefa é uma boa maneira de assegurar que as tarefas sejam completadas pela equipe. As tarefas podem ser atribuídas por email para cada responsável.

- *Standup meetings*

Em um ambiente de desenvolvimento FTS, *standup meetings* (reuniões diárias) podem ser facilmente adaptadas para a realização de sessões de *handoff*. As *standup meetings* são realizadas no início e no final de cada dia de trabalho. A equipe usa as sessões de *handoff* para discutir as tarefas que foram realizadas e as que devem ser continuadas ou iniciadas pelo próximo local de produção. As sessões de *handoff* devem durar no máximo 30 minutos, como previamente recomendado por Hess e Audy (2012).

- *Standup meetings* que antecedem finais de semana ou feriados

As *standup meetings* que antecedem finais de semana ou feriados também devem ser utilizadas para realizar o planejamento das tarefas que serão realizadas pela equipe que iniciará o dia de trabalho sem realizar a sessão de *handoff*. Em FTS, sessões de *handoff* podem ser inviabilizadas por finais de semana ou feriados. O uso de emails com informações detalhadas sobre o *status* do projeto e como o trabalho deve ser continuado contribui para minimizar possíveis problemas.

- Protocolo de comunicação

Equipes FTS podem enfrentar muitos problemas de comunicação, principalmente relacionados com os diferentes níveis de experiência, idiomas, cultura e fusos horários. Observou-se que para facilitar o entendimento, os membros da equipe devem falar devagar e claramente para reduzir sotaques. O uso de regras de comunicação tais como, a distribuição do tempo entre a equipe e a síntese do conteúdo no final de cada sessão de *handoff* pelos membros da equipe que estão recebendo a tarefa, pode contribuir para minimizar mal entendidos.

- Retrospectiva do *sprint*

Ao final do *sprint* 1 uma reunião foi realizada com toda a equipe para discutir as dificuldades enfrentadas e buscar soluções para a melhoria do processo de software desenvolvido para o projeto FTS. Também foi aplicado um questionário para os membros da equipe buscando avaliar as práticas de software desempenhadas pela equipe. Observou-se que a retrospectiva do *sprint* contribuiu para identificar problemas que ocorreram durante o *sprint*, mas que não foram tratados adequadamente. A equipe também discutiu novas soluções para o *sprint* 2. Além disso, também se observou que a retrospectiva do *sprint* também contribuiu para aumentar o nível de confiança entre a equipe.

Limitações

O estudo de caso possui limitações, as quais precisam ser destacadas. Primeiramente, foi estudado apenas um único projeto de software. Embora se observe o risco quanto à validade dos resultados obtidos, o desenvolvimento do projeto seguiu as regras para a aplicação de FTS e as práticas de software recomendadas pelo método Scrum.

A equipe alocada para o projeto possuía diferentes níveis de experiência. Para minimizar possíveis ameaças, sessões de treinamento foram conduzidas com toda a equipe antes do projeto iniciar. Também foram desenvolvidos tutoriais dando instruções de como o projeto de software seria desenvolvido, regras para o desenvolvimento de software FTS e práticas do método Scrum.

Especialistas no desenvolvimento de novas tecnologias e um *Scrum master* também foram alocados para o projeto visando revisar guias, práticas e processos e seguidos pela equipe FTS.

DISCUSSÃO

Na literatura foram encontrados estudos que discutem a aplicação dos métodos ágeis para FTS sobre quatro perspectivas diferentes. Enquanto Kroll et al. (2012) compara os benefícios das abordagens adaptativas e prescritivas para FTS, Carmel, Espinosa e Dubinsky (2009) mensuram a velocidade de trabalho de equipes distribuídas de acordo com a abordagem de desenvolvimento adotada. Por sua vez, Deny et al. (2008) aborda a utilização de práticas ágeis para a realização das sessões de *handoff*. Já Yap (2005) reporta a experiência obtida com a aplicação de XP para FTS. Outros estudos como o de Hess e Audy (2012), Tang et al (2011) e Carmel, Espinosa e Dubinsky (2010) também reportam a utilização dos métodos ágeis para FTS, mas não exploram o tema em profundidade.

Este estudo deu continuidade aos estudos já realizados examinando como os métodos ágeis podem colaborar para o desenvolvimento de software FTS. Observou-se com os resultados obtidos na literatura que os métodos ágeis permitem aumentar a velocidade do desenvolvimento colaborando com o principal objetivo do desenvolvimento de software FTS. Segundo Carmel, Dubinsky e Espinosa (2009), o FTS está focado na redução do tempo de desenvolvimento ou *time-to-market* e não oferece outros benefícios além deste.

O desenvolvimento do estudo de caso contribuiu para mostrar que é viável aplicar o método Scrum para o desenvolvimento FTS, por meio da adaptação de algumas práticas de software. Este resultado corrobora com o estudo realizado por Yap (2005). O estudo realizado por Yap (2005) também obteve um resultado similar, mas com a aplicação do método XP. Yap (2005) mostrou que o método XP pode ser usado por uma equipe distribuída desenvolvendo o software nas 24 horas contínuas de um dia de trabalho com uma base de dados compartilhada.

As lições aprendidas com o desenvolvimento do estudo de caso fornecem direções para uso efetivo do método Scrum para projetos FTS. Além disso, com base nas lições aprendidas, novas práticas e processos de software podem ser criados para FTS.

CONCLUSÃO E TRABALHOS FUTUROS

Este estudo examinou a aplicação dos métodos ágeis para o desenvolvimento de software FTS. Foram identificados os principais estudos que abordam a aplicação dos métodos ágeis para FTS e os resultados obtidos. Também foi desenvolvido um estudo de caso, o qual constou do desenvolvimento de um projeto de software no modo FTS com a adoção do método ágil Scrum.

Na literatura de FTS pode-se verificar que o uso dos métodos ágeis tem se mostrado promissor. Os estudos identificados ainda visam principalmente examinar a adequação dos métodos ágeis para FTS. Baseado nos resultados da literatura pode-se inferir que a adoção dos métodos ágeis para FTS ainda é campo de estudo imaturo com muitas oportunidades de pesquisa a serem exploradas.

A experiência obtida com o desenvolvimento do estudo de caso mostrou a viabilidade do método Scrum para o desenvolvimento de software FTS. Destaca-se um conjunto de seis lições aprendidas, as quais contribuirão para o desenvolvimento de estudos futuros.

Nesse sentido, trabalhos futuros continuarão investigando como os métodos ágeis podem colaborar para o desenvolvimento de software FTS. Também serão investigadas as dificuldades enfrentadas por empresas para desenvolver projetos FTS baseado em práticas ágeis. Pretende-se propor práticas e processos de software inspirados no desenvolvimento de software ágil para melhor apoiar e colaborar com o desenvolvimento de software FTS.

AGRADECIMENTOS

Os autores deste estudo agradecem ao apoio financeiro dado pela Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) através do Programa Ciência sem Fronteiras e ao programa PDTI, financiado pela Dell Computadores do Brasil Ltda, com recursos da Lei Federal Brasileira número 8.248/91. Os autores também agradecem ao centro de pesquisa Lero (*The Irish Software Engineering Research Centre*).

REFERÊNCIAS

1. Bannerman, P. L., Hossain, E. and Jeffery, R. (2012) *Scrum Practice Mitigation of Global Software Development Coordination Challenges: A Distinctive Advantage?*, *System Science (HICSS)*, 2012 45th Hawaii International Conference on , pp.5309-5318, 4-7.
2. Cameron, A. (2004) A Novel Approach to Distributed Concurrent Software Development using a Follow-the-Sun Technique, *Unpublished EDS working paper*.
3. Carmel, E. (1999) *Global Software Teams: collaborating across borders and time zones*. Published by Prentice Hall-PTR.
4. Carmel, E. and Espinosa, J. A. (2011) *I'm Working While They're Sleeping: Time Zone Separation Challenges and Solutions*, Kindle Edition, 188 p.
5. Carmel, E., Espinosa, A. and Dubinsky, Y. (2009) Follow the Sun Software Development: New Perspectives, Conceptual Foundation, and Exploratory Field Study, *42nd Hawaii International Conference on System Sciences, Proceedings*.
6. Carmel, E., Espinosa, J. and Dubinsky, Y. (2010) Follow the Sun Workflow in Global Software Development, *Journal of Management Information Systems*, Vol. 27 No. 1, 17 – 38.
7. Casey, V., Deshpande S. and Richardson, I. (2008) Outsourcing Software Development the Remote Project Manager's Perspective, *The 2nd Global Sourcing Workshop - Services, Knowledge and Innovation*, March 10th to 13th, Val D'Isere, France.
8. Denny, N., Crk, I. Nadella, R. S. and Gupta, A. (2009) Agile Software Processes for the 24-Hour Knowledge Factory Environment, February 27, Disponível em SSRN: <http://ssrn.com/abstract=1017184>.
9. Denny, N., Mani, S., Nadella, R. S., Swaminathan, M. and Samdal, J. (2008) Hybrid Offshoring: Composite Personae and Evolving Collaboration Technologies, *IRMJ* 21(1): 89-104.
10. Deshpande, S. and Richardson, I. (2009) Management at the Outsourcing Destination - Global Software Development in India, *International Conference on Global Software Engineering (ICGSE)*, Washington, DC, USA, 217-22.
11. Gupta, A., Hu, L., Hedberg, T., Prendergast, C. and Crk, I. (2012) Creating the 24-Hour Knowledge Factory (February 13). Disponível em: <http://ssrn.com/abstract=2004791>.

12. Hess, E. and Audy, J. L. N. (2012) FTSProc: a Process to Alleviate the Challenges of Projects that Use the Follow-the-Sun Strategy, *International Conference on Global Software Engineering (ICGSE)*, Porto Alegre, Brazil.
13. Jalali, S. and Wohlin, C. (2010) Agile Practices in Global Software Engineering: A Systematic Map, *International Conference on Global Software Engineering (ICGSE)*, pp. 45-54, Princetown, USA.
14. Kircher, M., Jain, P., Corsaro, A. and Levine, D. (2011) Distributed eXtreme Programming, *Proceedings of XP Conference*, Cagliari, Villasimius, Sardinia, Italy.
15. Kroll, J., Santos, A. R., Prikladnicki, R., Hess, E. R., Glanzner, R., Sales, A., Audy, J. L.N and Fernandes, P. (2012) Follow-the-Sun Software Development: A Controlled Experiment to Evaluate the Benefits of Adaptive and Prescriptive Approaches, *24th International Conference on Software Engineering & Knowledge (SEKE 2012)*, 551-556.
16. Layman, L., Williams, L., Damian, D. and Bures, H. (2006) Essential communication practices for extreme programming in a global software development team, *Information and Software Technology*, vol. 48, no. 9, 781–794.
17. Niinimäki, T. (2011) Face-to-Face, Email and Instant Messaging in Distributed Agile Software Development Project, *Sixth International Conference on Global Software Engineering Workshop (ICGSEW)*, p.78-84.
18. Paasivaara, M., Durasiewicz, S. and Lassenius, C. (2008) Distributed Agile Development: Using Scrum in a Large Project, *International Conference on Global Software Engineering (ICGSE)*, pp.87-95, 17-20.
19. Sato, G. Y., Huzita, E. H. M. and Leal, G. C. L. (2011) A process engine for outsourced software development, *Information Systems and Technologies (CISTI), 2011 6th Iberian Conference on*, vol., p.1-4.
20. Šmite, D., Moe, N. B. and Ågerfalk, P. J. (2010) Agility Across Time and Space: Implementing Agile Methods in Global Software Projects. 1st Edition, XXXVI, 341 p. 37 illus.
21. Solingen, V. R. and Valkema, M. (2010) The Impact of Number of Sites in a Follow the Sun Setting on the Actual and Perceived Working Speed and Accuracy: A Controlled Experiment, *5th International Conference on Global Software Engineering (ICGSE)*, , 165- 174.
22. Sutherland, J., Viktorov, A., Blount, J. and Puntikov, N. (2007) Distributed Scrum: Agile Project Management with Outsourced Development Teams. *System Sciences, HICSS 2007. International Conference on*, p.274, 2007.
23. Tang, J. C., Zhao, C., Cao, X. and Inkpen, K. (2011) Your time zone or mine?: a study of globally time zone-shifted collaboration, *Proceedings of the ACM 2011 conference on Computer supported cooperative work (CSCW '11)*. ACM, New York, NY, USA, 235-244.
24. Treinen, J. J. and Miller-Frost, S. L. (2006) Following the Sun: Case Studies in Global Software Development, *IBM Systems Journal*, Volume 45, Number 4.
25. Visser, C. and Solingen, V. R. (2009) Selecting Locations for Follow-the Sun Software Development: Towards A Routing Model, *Fourth International Conference on Global Software Engineering (ICGSE)*.
26. Yap, M. (2005) Follow the sun: distributed extreme programming development, *Agile Conference Proceedings*, 218-224.